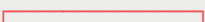




ER RELATIONSHIPS VERSION 2

Symbol Meanings:

 = Strong Entity (single rectangle)

 = Weak Entity (double rectangle)

 = Relationship (diamond)

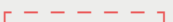
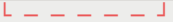
 = Weak Relationship (double diamond)
connects TO a weak entity

 = Partial Participation (single line)
"this side is OPTIONAL"

 = Total Participation (double line)
"this side is MANDATORY"

1 or N = Cardinality label written ON the line

 = Generalization/Specialization
(circle with d = disjoint, T = total)

 = Aggregation box (dashed rectangle)
 drawn around a relationship cluster

Overall View of schema:

ZONE A – TOP HALF
The Supply Side
(Vendor, Transport, Hotel, Package)
Everything vendors CREATE and LIST
ZONE B – MIDDLE
The Transaction Layer
(User, Booking, Generalization, Aggregation box)
Everything that happens when someone BOOKS
ZONE C – BOTTOM HALF
The Features Layer
(Insurance, Refund, Review, Promo, DemandTag)
Everything that happens AFTER booking

ZONE A — THE SUPPLY SIDE

Drawing Steps 1 through 9

▶ STEP 1 — Draw Vendor

Draw one rectangle in the top left of Zone A. Write **Vendor** inside it.

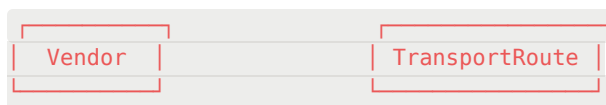


That's it for now. Nothing connects to it yet. It just sits there.

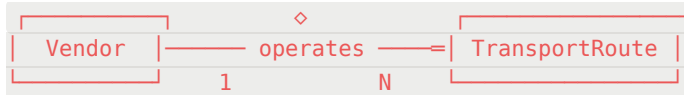
Why strong? IndiGo Airlines can be registered on your platform without having listed a single flight yet. It exists independently.

▶ STEP 2 — Vendor OPERATES TransportRoute

Now draw a second rectangle to the right of Vendor. Write **TransportRoute** inside it. Between them draw a diamond. Write **operates** inside the diamond.



Now connect them through the diamond:



Cardinality:

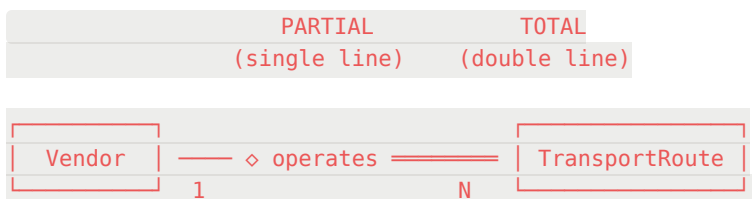
- One Vendor operates MANY TransportRoutes → write **1** near Vendor, **N** near TransportRoute

Participation — Vendor side (single line):

IndiGo can be registered on your platform without having listed any route yet. So Vendor's participation is **partial**. Draw a single line from Vendor to the diamond.

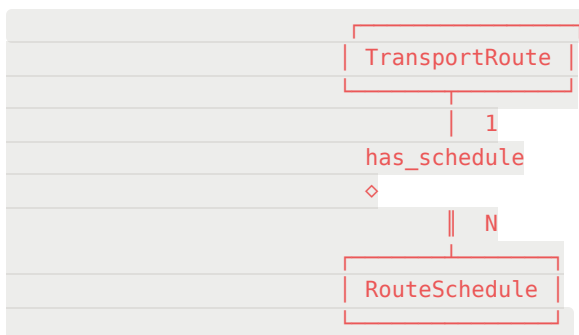
Participation — TransportRoute side (double line):

A route like "Mumbai → Delhi" CANNOT exist on your platform without belonging to some vendor. Who operates it? Unknown = invalid. So TransportRoute's participation is **total**. Draw a double line from the diamond to TransportRoute.



STEP 3 — TransportRoute HAS_SCHEDULE RouteSchedule

Draw **RouteSchedule** below TransportRoute. Connect with diamond **has_schedule**.



Cardinality:

One TransportRoute has MANY RouteSchedules. The Mumbai → Delhi route has a 6AM flight and a 9PM flight — two different schedules. → **1:N**

Participation — TransportRoute side (single line):

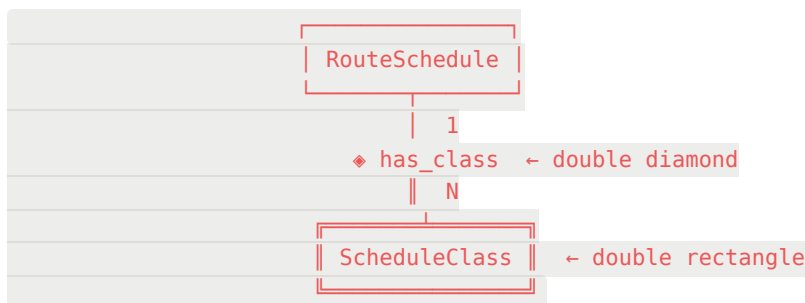
A route can exist in the system before any schedule is added to it. Partial → single line.

Participation — RouteSchedule side (double line):

A schedule like "departs 6AM, arrives 8:15AM, runs Mon/Wed/Fri" means absolutely nothing without knowing WHICH route it belongs to. It cannot exist alone. Total → double line.

▶ STEP 4 — RouteSchedule HAS_CLASS ScheduleClass

ScheduleClass is a weak entity. Draw it as a double rectangle.



Why double diamond (weak relationship)?

Because `has_class` connects a strong entity to a WEAK entity. Whenever a relationship's child is a weak entity, the diamond itself becomes a double diamond. This is the rule — no exceptions.

Cardinality: One RouteSchedule has MANY ScheduleClasses → **1:N**

Participation — RouteSchedule side (single line):

A schedule can exist before classes are defined on it. Partial.

Participation — ScheduleClass side (double line):

Economy class with 180 seats at ₹3500 means nothing without knowing which schedule it belongs to. Total. And since it's a weak entity, double line is automatic.

▶ STEP 5 — THE TERNARY: generates → ScheduleDeparture

This is the most important relationship in Zone A. Read carefully before drawing.

Why ternary?

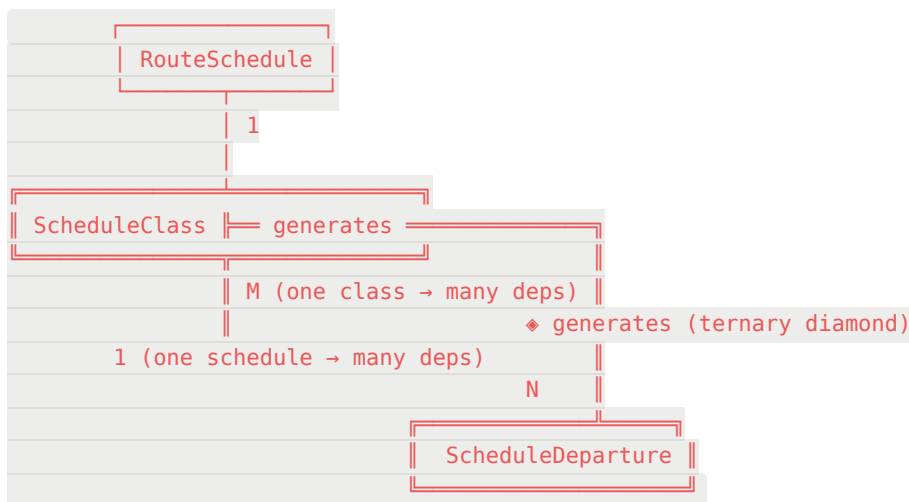
A `ScheduleDeparture` is one real bookable slot. To define it you need ALL THREE of:

- WHICH schedule? (6AM Mumbai → Delhi)
- WHICH class? (Economy)
- WHICH date? (15th March 2025)

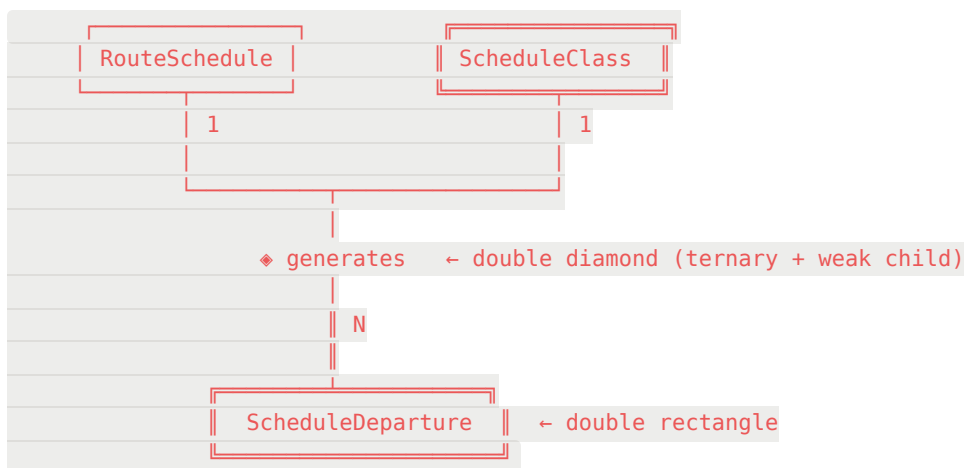
Remove any one of these three and the departure is undefined. You cannot split this into two binary relationships without losing meaning. This is a genuine ternary.

ScheduleDeparture is a weak entity — it cannot exist without its three parents.

Draw it like this — diamond in the centre, three lines radiating out:



Let me draw this more cleanly as it actually looks on paper:



Cardinality on all 3 lines:

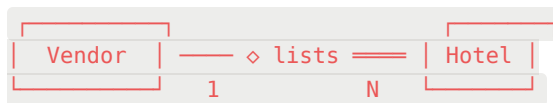
- RouteSchedule → generates: **1** (one schedule participates in many departures)
- ScheduleClass → generates: **1** (one class participates in many departures)
- generates → ScheduleDeparture: **N** (many specific date-slots are generated)

Participation:

- RouteSchedule: partial (single line) — a schedule exists before any departure dates are generated
- ScheduleClass: partial (single line) — a class exists before any departure dates are generated
- ScheduleDeparture: total (double line) — it MUST have both a schedule and a class, it's weak

▶ STEP 6 — Vendor LISTS Hotel

Draw **Hotel** in the top right area of Zone A. Connect to Vendor.



Cardinality: One Vendor lists MANY Hotels → **1:N**

Participation — Vendor side (single line):

A vendor (transport company) may never list a hotel. Partial.

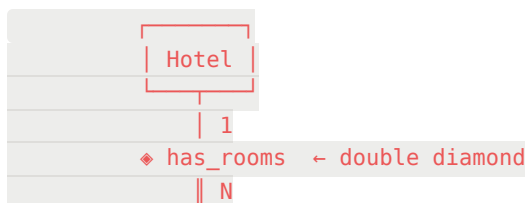
Participation — Hotel side (double line):

A hotel property cannot float in the system without a vendor owning it. Total.

▶ STEP 7 — Hotel HAS_ROOMS RoomCategory → RoomAvailability

Two relationships drawn in sequence downward from Hotel.

First: Hotel → RoomCategory

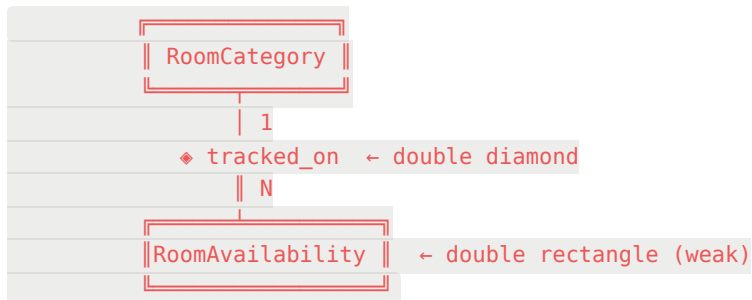




Cardinality: One Hotel has MANY RoomCategories → **1:NHotel side:** partial (single line) — hotel exists before room types are defined

RoomCategory side: total (double line) — room type is meaningless without the hotel

Second: RoomCategory → RoomAvailability



Cardinality: One RoomCategory tracked across MANY dates →

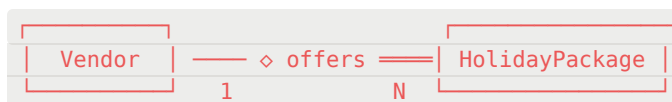
1:NRoomCategory side: partial — a room category exists before availability is entered

RoomAvailability side: total — "3 rooms available on 20th March" is meaningless without knowing which room type

▶ STEP 8 — Vendor OFFERS HolidayPackage → PackageDeparture

Two relationships drawn downward from Vendor.

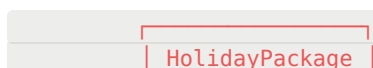
First: Vendor → HolidayPackage

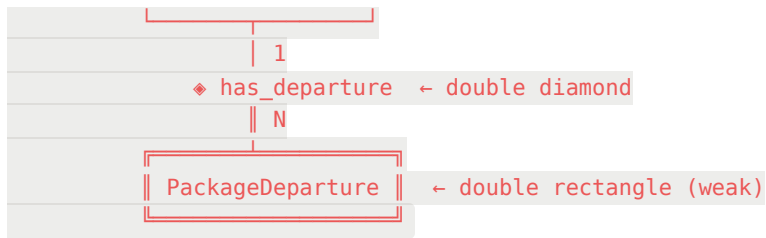


Cardinality: One Vendor offers MANY HolidayPackages → **1:NVendor side:** partial — a vendor can exist without any package listed

HolidayPackage side: total — a package must belong to some vendor

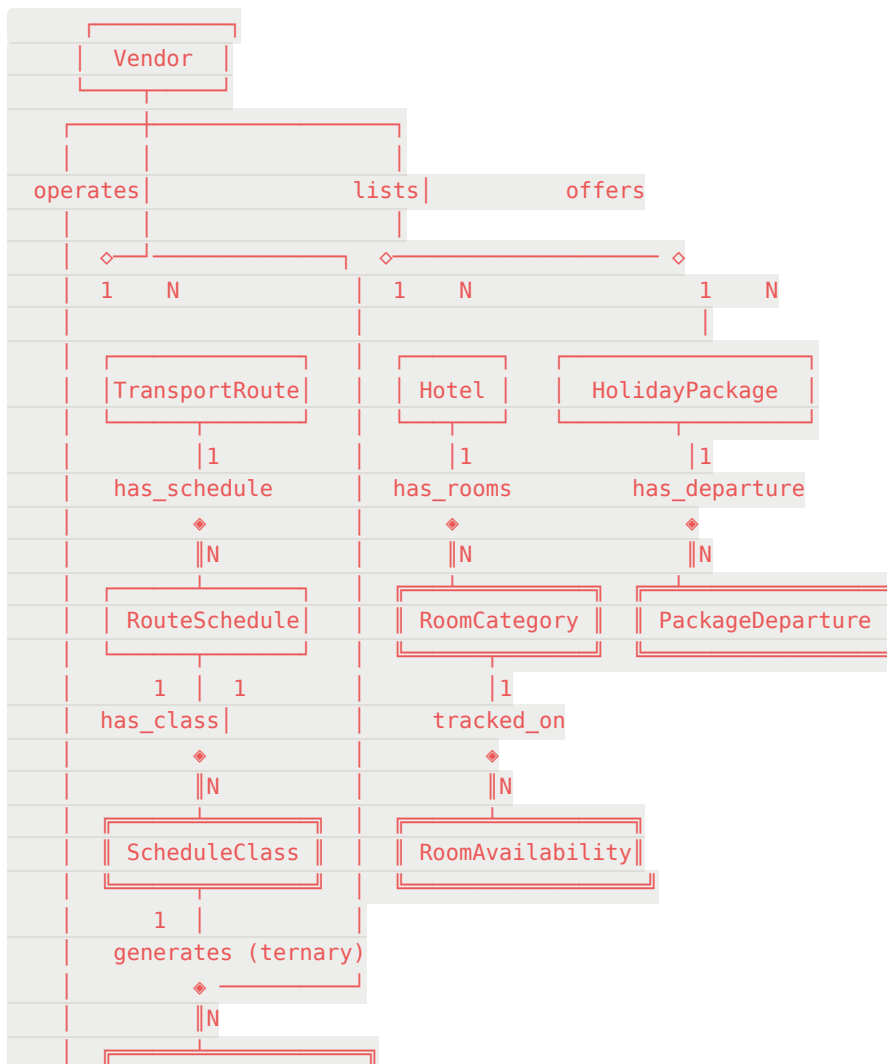
Second: HolidayPackage → PackageDeparture

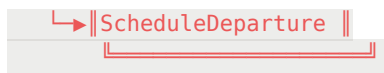




Cardinality: One package has MANY departure slots → **1:N HolidayPackage side:** partial — package exists before departure dates are added
PackageDeparture side: total — a departure slot is meaningless without its package

ZONE A COMPLETE — What Your Paper Should Look Like





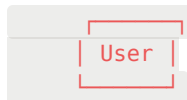
ZONE B — THE TRANSACTION LAYER

Drawing Steps 9 through 14

This is the heart of the diagram. Draw it in the middle of your paper with plenty of space.

▶ STEP 9 — Draw User

Draw one rectangle in the centre-left of Zone B.



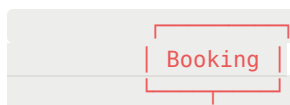
Why strong? A user can register and never book anything. Exists independently.

▶ STEP 10 — Generalization/Specialization of Booking

Draw this BEFORE drawing any booking relationships. The superclass/subclass structure must exist before anything connects to it.

Here is the exact sequence of shapes:

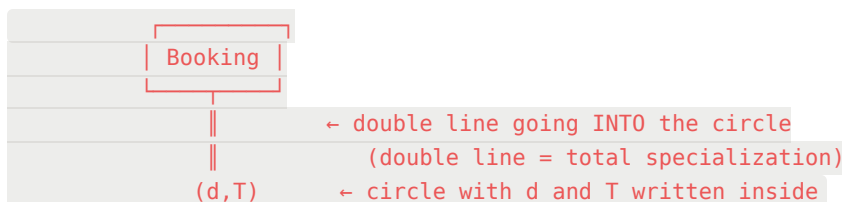
① Draw the superclass — Booking



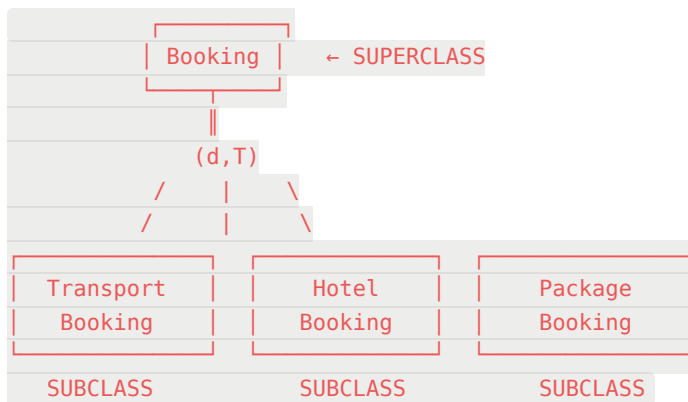
② Draw a circle below Booking — write d,T inside it

d = disjoint (a booking is exactly ONE type, never two simultaneously)

T = total (every booking MUST be one of the three types, no generic booking allowed)



③ Draw three lines going DOWN from the circle to three subclasses



What this means in plain English:

Every row in Booking is either a TransportBooking OR a HotelBooking OR a PackageBooking. Never two at once (disjoint). And every Booking row must be one of these three — there's no orphan booking that belongs to none (total).

What each subclass inherits + adds:

Booking (superclass) owns:

→ booking_id, user_id, booked_at, status,
base_amount, discount_amount, final_amount,
payment_method, payment_ref, booking_type

TransportBooking ADDS:

→ departure_id
→ num_seats

HotelBooking ADDS:

→ hotel_id
→ room_category_id
→ checkin_date
→ checkout_date
→ num_rooms
→ num_guests

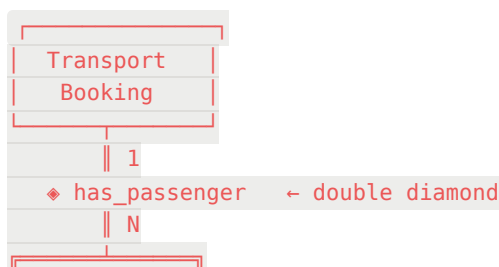
PackageBooking ADDS:

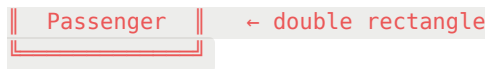
→ package_departure_id
→ num_travellers

▶ STEP 11 — Weak entities hanging off subclasses

Two weak entities connect directly to the subclasses, NOT to the superclass Booking.

Passenger hangs off TransportBooking:

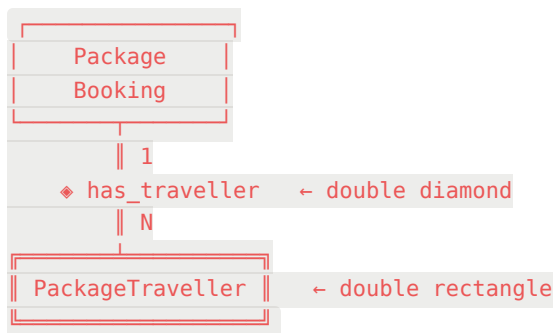




Why total on both sides here?

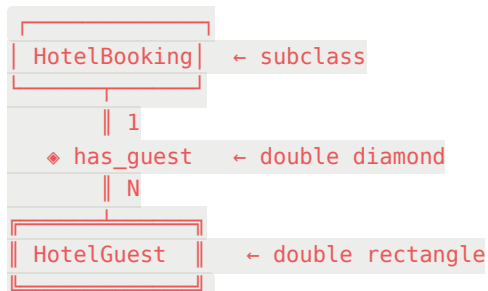
- TransportBooking side: double line — a transport booking MUST have at least one passenger. You cannot book a flight for nobody.
- Passenger side: double line — a passenger record MUST belong to a transport booking. An orphan passenger means nothing.

PackageTraveller hangs off PackageBooking:



Same reasoning — a package booking must have at least one traveller, and a traveller record must belong to a package booking.

Hotel_Guests hangs off HotelBooking



▶ STEP 12 — The Three Ternary Booking Relationships

Now connect User and the supply-side entities to the Booking superclass through ternary relationships.

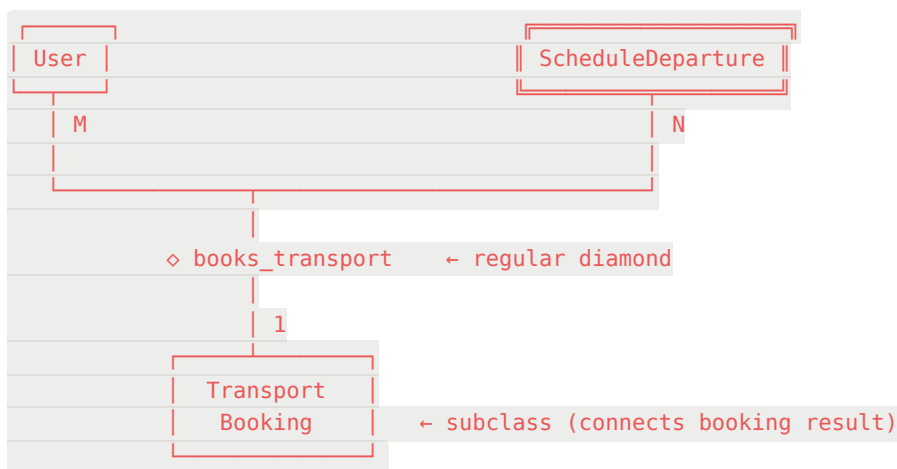
Why ternary for booking?

When someone books something, THREE things are involved simultaneously:

- WHO is booking (User)
- WHAT they are booking (ScheduleDeparture / RoomCategory / PackageDeparture)
- The RESULT of that booking (TransportBooking / HotelBooking / PackageBooking)

You cannot model this as two separate binary relationships without losing which booking belongs to which user AND which service.

Ternary 1 — books_transport



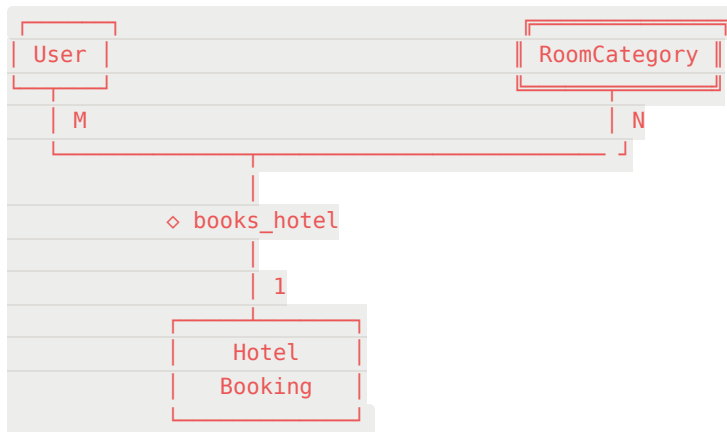
Cardinality:

- User : `books_transport` → **M** (one user makes many transport bookings)
- ScheduleDeparture : `books_transport` → **N** (one departure booked by many users)
- `books_transport` → TransportBooking → **1** (each booking event produces one booking record)

Participation:

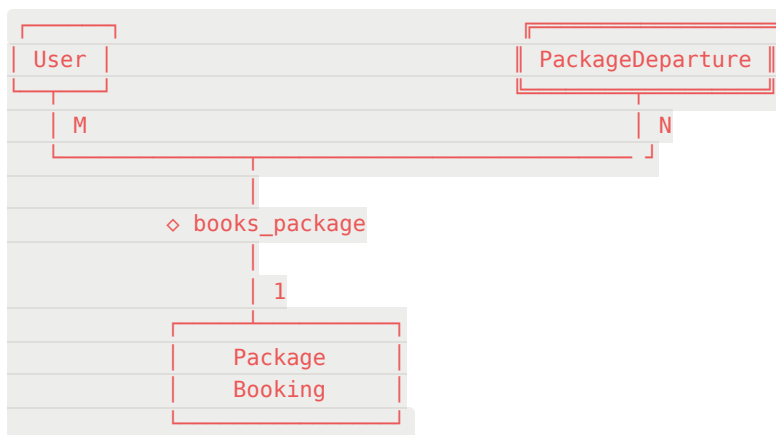
- User: partial (single line) — user may have no transport bookings
- ScheduleDeparture: partial (single line) — a departure may have zero bookings yet
- TransportBooking: total (double line) — a transport booking record MUST come from this relationship

Ternary 2 — books_hotel



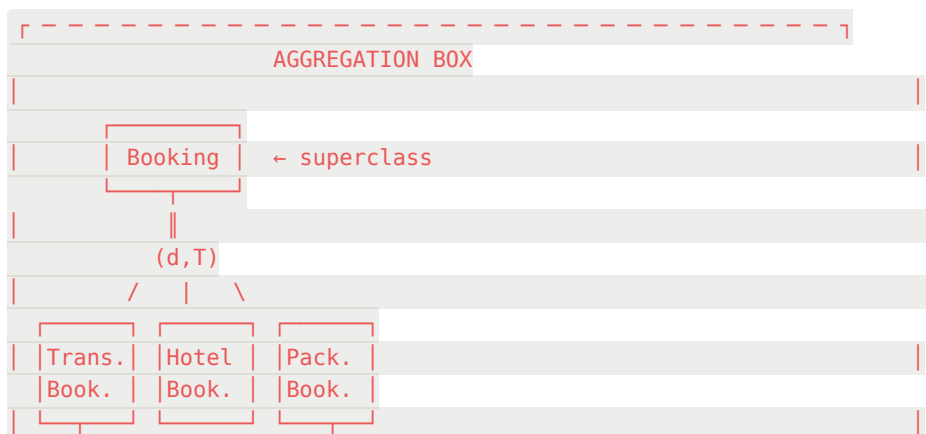
Same cardinality logic. Same participation logic.

Ternary 3 — books_package



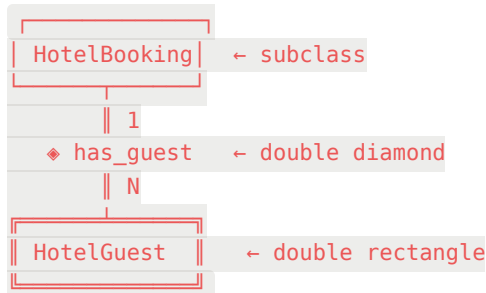
▶ STEP 13 — THE AGGREGATION BOX

Now draw a large **dashed rectangle** around this entire cluster:





In this add hotel guest as well



Why this box exists:

Without the aggregation box, the features in Zone C (Insurance, Refund, Review) would have to connect to some individual entity — but they don't connect to just User, or just ScheduleDeparture. They connect to the ENTIRE booking event. The dashed box says: *"treat everything inside me as one single thing."*

The entities in Zone C will draw their relationship lines to the **border of this dashed box**, not to any specific entity inside it.

Label the box clearly: write **"BOOKING AGGREGATION"** along the top dashed border.

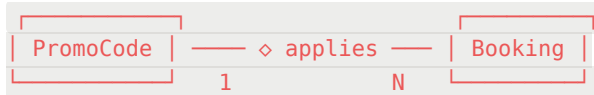
ZONE C — THE FEATURES LAYER

Drawing Steps 14 through 17

Draw these below the aggregation box, in Zone C.

▶ STEP 14 — PromoCode APPLIES to Booking

PromoCode connects to the Booking superclass (before the subclasses split). A promo is applied at booking time, before we even know the full details.



Cardinality: One PromoCode applies to MANY Bookings → **1:N**

PromoCode side: partial (single line)

A promo code can be created and never used by anyone. It exists in the system waiting.

Booking side: partial (single line)

Most bookings won't use a promo code at all. Optional.

▶ STEP 15 — Booking COVERED_BY Insurance

This relationship line starts from the **border of the aggregation box** and goes to Insurance.



Cardinality: One Booking has at most ONE Insurance record → **1:1**

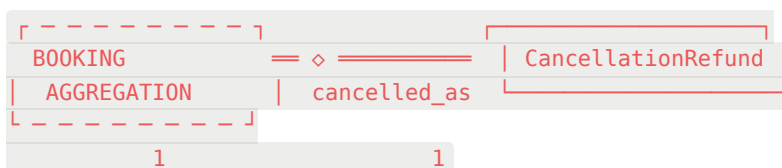
Aggregation box side: partial (single line)

Most bookings will never have insurance added. Optional.

Insurance side: total (double line)

An insurance record **MUST** belong to some booking. An insurance policy floating with no booking attached is meaningless.

▶ STEP 16 — Booking CANCELLED_AS CancellationRefund



Cardinality: One Booking has at most ONE CancellationRefund → **1:1**

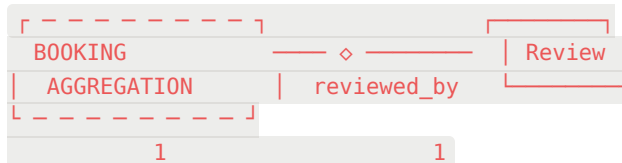
Aggregation box side: partial (single line)

Most bookings are never cancelled. Optional.

CancellationRefund side: total (double line)

A refund record MUST belong to a booking. Cannot exist alone.

▶ STEP 17 — Booking REVIEWED_BY Review



Cardinality: One Booking has at most ONE Review → 1:1

Aggregation box side: partial (single line)

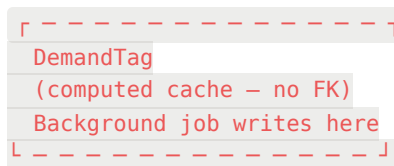
Most bookings won't get reviewed. Optional.

Review side: total (double line)

A review must come from a real booking. Enforces authenticity — you can only review something you actually booked.

▶ STEP 18 — DemandTag (Standalone)

Draw DemandTag as a rectangle with a dashed border, in the bottom corner of Zone C. No relationship lines connect to it.



Write a small annotation next to it: *"Populated by background job. Loose reference via service_type + service_id. No FK constraint."*

COMPLETE MASTER CHECKLIST

Go through every single item before calling the diagram done.

Rectangles — 23 total

Entity	Type	Zone
User	Single rectangle	B
Vendor	Single rectangle	A
TransportRoute	Single rectangle	A
RouteSchedule	Single rectangle	A
Hotel	Single rectangle	A
RoomCategory	Single rectangle	A
HolidayPackage	Single rectangle	A
PromoCode	Single rectangle	C
Booking	Single rectangle (superclass)	B
TransportBooking	Single rectangle (subclass)	B
HotelBooking	Single rectangle (subclass)	B
PackageBooking	Single rectangle (subclass)	B
Insurance	Single rectangle	C
CancellationRefund	Single rectangle	C
Review	Single rectangle	C
DemandTag	Dashed rectangle	C
ScheduleClass	Double rectangle	A
ScheduleDeparture	Double rectangle	A
PackageDeparture	Double rectangle	A
RoomAvailability	Double rectangle	A
Passenger	Double rectangle	B
PackageTraveller	Double rectangle	B
HotelGuest	Double Rectangle	C

Diamonds — 18 total

Relationship	Type	Cardinality
operates	Regular ◇	1:N
has_schedule	Regular ◇	1:N

Relationship	Type	Cardinality
has_class	Double ◆	1:N
generates (ternary)	Double ◆	1:1:N
lists	Regular ◇	1:N
has_rooms	Double ◆	1:N
tracked_on	Double ◆	1:N
offers	Regular ◇	1:N
has_departure	Double ◆	1:N
books_transport (ternary)	Regular ◇	M:N:1
books_hotel (ternary)	Regular ◇	M:N:1
books_package (ternary)	Regular ◇	M:N:1
has_passenger	Double ◆	1:N
has_traveller	Double ◆	1:N
applies	Regular ◇	1:N
covered_by	Regular ◇	1:1
cancelled_as	Regular ◇	1:1
reviewed_by	Regular ◇	1:1

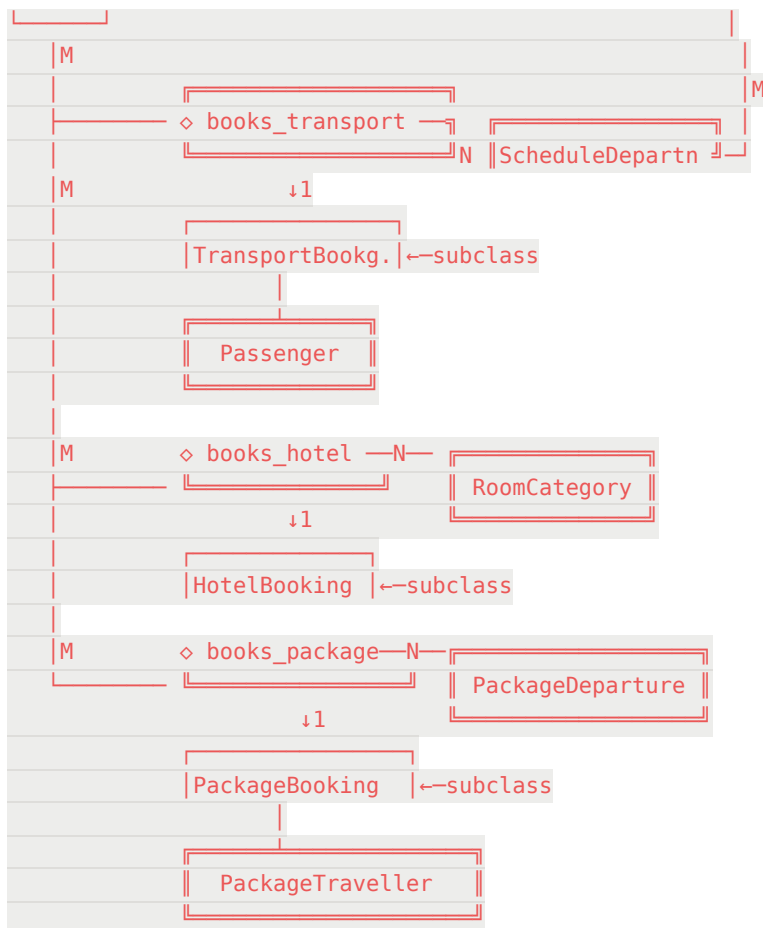
Special Symbols — verify all 3

Symbol	Drawn?	Location
Generalization circle with (d,T)	○	Below Booking superclass in Zone B
Double line from Booking to circle	=	Total specialization
Aggregation dashed box	⌈	Around entire Booking cluster in Zone B

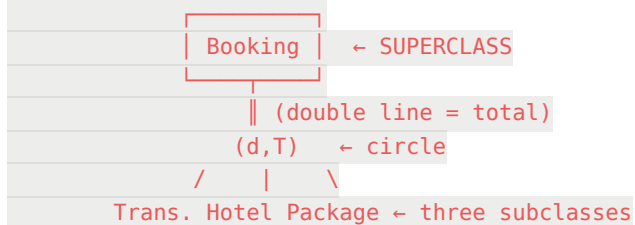
THE COMPLETE STORY IN ONE PICTURE

ZONE A

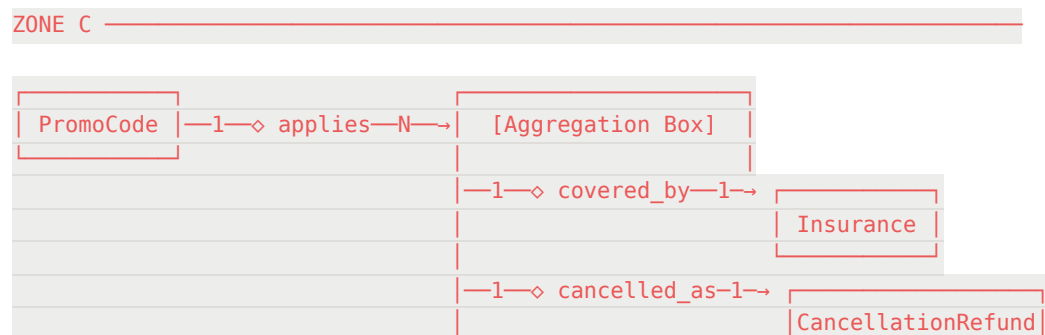
—◇ operates —1—N—

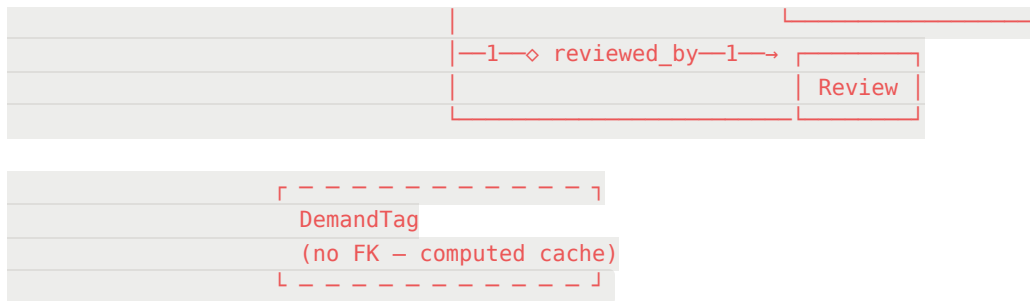


ALL THREE SUBCLASSES sit under:



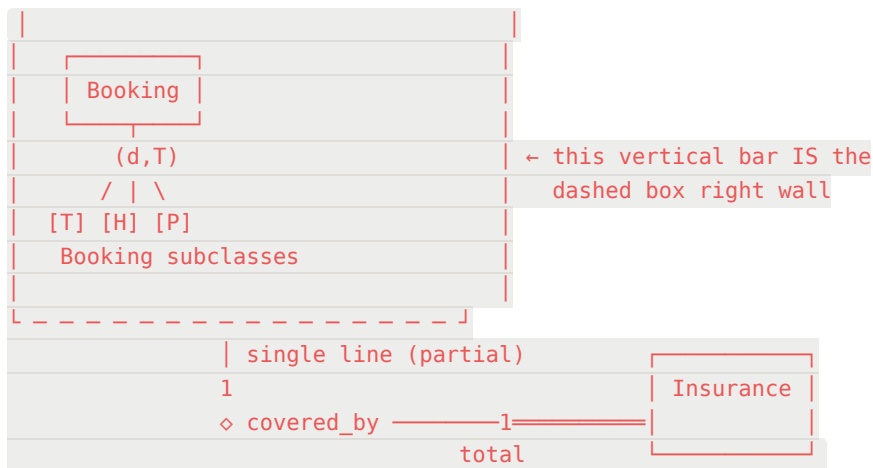
BOOKING AGGREGATION BOX (dashed rectangle around all above)



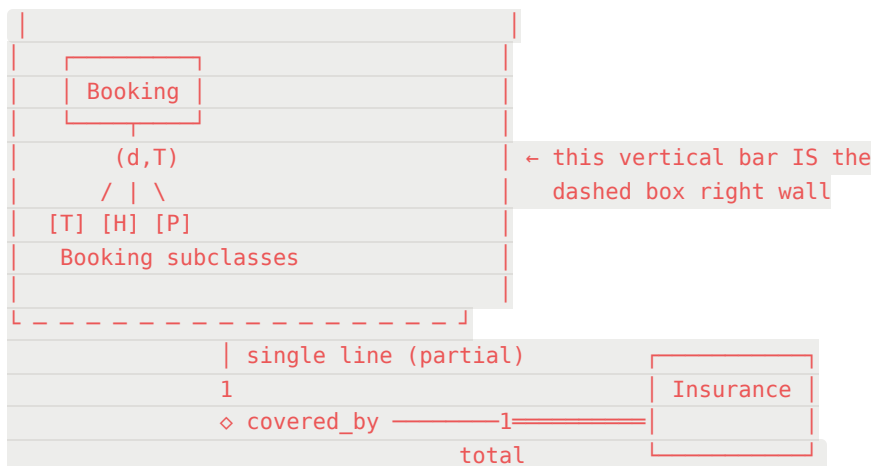


How to use Booking aggregation as an entity :

Visually on paper it looks like this — the diamond floats between the box and the entity:



Visually on paper it looks like this — the diamond floats between the box and the entity:



Visually on paper it looks like this — the diamond floats between the box and the entity:

